

Distributed Systems Coordination: Mutual Exclusion, Snapshots, and Leader Election

This study guide provides a comprehensive review of coordination in distributed systems, focusing on the challenges of achieving mutual exclusion, the mechanisms for electing leaders, the logic behind distributed snapshots, and the role of modern coordination services.

Part 1: Short-Answer Quiz

Instructions: Answer the following questions using 2–3 sentences.

1. **Why is implementing shared-resource locking difficult in a distributed system compared to a single-machine environment?**
2. **What is the primary trade-off when using a centralized approach for mutual exclusion?**
3. **How does the Token-ring approach for mutual exclusion ensure fairness?**
4. **What is the message complexity of the Ricart-Agrawala algorithm per request?**
5. **What is the "Bully algorithm" approach to leader election?**
6. **Why can taking local snapshots at different wall-clock times lead to inconsistent results?**
7. **What is the primary role of "marker" messages in the Chandy-Lamport snapshot algorithm?**
8. **What was the original goal of the ZooKeeper service developed by Yahoo?**
9. **What are the two simple but powerful primitives used by ZooKeeper to manage coordination?**
10. **How has the coordination architecture of Apache Kafka evolved in recent versions?**

Part 2: Quiz Answer Key

1. **Distributed Mutual Exclusion Challenges:** Distributed systems lack shared memory and a global clock, making it impossible to use traditional local locking mechanisms. Furthermore, messages can be lost or delayed, and processes can crash, requiring robust algorithms to ensure safe resource access.
2. **Centralized Approach Trade-offs:** The centralized approach is simple to implement because a single coordinator grants all locks. However, it suffers from being a single point of failure; if the coordinator crashes, the entire locking system fails.

3. **Token-ring Fairness:** In this approach, a token circulates through the processes in a logical ring, and only the holder of the token can access the resource. This ensures fairness and prevents starvation, though the loss of the token is a significant failure state.
4. **Ricart-Agrawala Complexity:** The Ricart-Agrawala algorithm is a fully distributed, timestamp-based voting approach. It requires $2(N-1)$ messages for every request to enter a critical section, where N is the number of processes.
5. **Bully Algorithm:** The Bully algorithm is a classic leader election method based on process identifiers. In this "intuition-only" model, the process with the highest unique ID is elected as the leader.
6. **Inconsistent Snapshots:** Because messages are often in flight, taking snapshots at different wall-clock times can make money or data appear to disappear or double-count. This occurs because the snapshots do not represent a single logical moment across the entire distributed system.
7. **Chandy-Lamport Markers:** Markers are sent on all outgoing channels to initiate a snapshot; when a process receives its first marker, it records its local state. It then begins recording messages on other channels until markers have been received on all channels, completing the consistent cut.
8. **ZooKeeper's Original Goal:** ZooKeeper was originally developed at Yahoo to provide reliable coordination for large-scale distributed systems. It was designed to handle tasks like task assignment and configuration management.
9. **ZooKeeper Primitives:** ZooKeeper utilizes "znodes" (a hierarchical namespace similar to a file system) and "watches" (a mechanism to notify clients of changes). Together, these allow developers to build complex coordination logic.
10. **Kafka Evolution:** Historically, Kafka relied on ZooKeeper for managing its coordination and metadata. In modern implementations, Kafka has largely moved away from ZooKeeper in favor of KRaft (Kafka Raft), its own internal Raft-based consensus implementation.

Part 3: Essay Questions

Instructions: Use the provided source material to outline or draft responses for the following topics. (Answers not provided).

1. **The Evolution of Mutual Exclusion:** Compare and contrast the Centralized, Token-ring, and Ricart-Agrawala approaches. Discuss the evolution from these classic algorithms to the use of modern coordination services like ZooKeeper or etcd.
2. **The Mechanics of a Consistent Cut:** Explain why "photographing a moving system" is difficult in distributed environments and describe how the Chandy-Lamport algorithm ensures that in-transit messages are correctly captured in a global snapshot.
3. **Leader Election in Production Systems:** Discuss the necessity of a single coordinator in distributed tasks such as replication and task assignment. Compare classic algorithms like the Bully and Ring algorithms with modern standards like Raft and ZooKeeper.

4. **The Role of Coordination Services:** Analyze the importance of ZooKeeper in the Hadoop ecosystem. Explain the significance of the ZAB protocol and why systems like Kafka are eventually migrating toward internal implementations like KRaft.
5. **Logical vs. Physical Time in Coordination:** Based on the lab previews, discuss why vector clocks and clock synchronization (NTP-style) are critical for ordering events and how they support later replication and consensus tasks.

Part 4: Glossary of Key Terms

Term	Definition
Bully Algorithm	A classic leader election algorithm where the process with the highest ID is chosen as the coordinator.
Centralized Mutex	A mutual exclusion approach where a single coordinator grants locks; it is simple but represents a single point of failure.
Chandy-Lampert	A distributed snapshot algorithm that uses marker messages to create a consistent global state, including in-transit messages.
Consistent Cut	A view of a distributed system's global state at a single logical moment where no recorded message is received before it is sent.
KRaft	Kafka Raft; a modern implementation used by Kafka to handle its own coordination without relying on ZooKeeper.
Mutual Exclusion	A requirement that only one process can access a shared resource (like a file or database row) at a time.
Ricart-Agrawal a	A fully distributed mutual exclusion algorithm that uses timestamp-based voting and requires $2(N-1)$ messages per request.

Ring Algorithm	A leader election method where a token or message circulates through a logical ring of processes.
Token-ring	A mutex approach where a token circulates; the process holding the token has the right to access the resource.
Vector Clock	An implementation used to track the ordering of events and determine if they are before, after, or concurrent in a distributed system.
ZAB	ZooKeeper Atomic Broadcast; the underlying protocol used by ZooKeeper, similar in spirit to the Raft consensus algorithm.
znodes	The basic data nodes in ZooKeeper's hierarchical namespace used as primitives for coordination.
ZooKeeper	A coordination service originally developed by Yahoo to provide reliable synchronization and configuration management for distributed systems.